MeshSOS

Semantic Compression for Disaster Mesh Networks

Complete Project Report — what is built, what is missing,
and what to do next

27 October 2026

How to read this. Section 1 explains the project in plain words. Sections 2–4 describe what exists today. Sections 5–8 cover the models, data and results. Section 9 is the honest list of what is broken or missing and how to fix each item. Section 10 is the schedule. If you only read one section, read Section 9.

Contents

1 What this project is **3**
- 1.1 The problem . . . 3
- 1.2 The specific problem we work on . . . 3
- 1.3 Why splitting a message is bad . . . 3
- 1.4 What we built . . . 3

2 Where the project stands today **4**

3 The application, explained **4**
- 3.1 Who the actors are . . . 4
- 3.2 What the user actually does . . . 4
- 3.3 Technologies used . . . 5
- 3.4 How data flows, step by step . . . 5
- 3.5 What the 3 bytes contain . . . 5

4 Existing research and where the gaps are **6**
- 4.1 What already exists . . . 6
- 4.2 The gaps we found . . . 6
- 4.3 What is new in our work . . . 6

5 The models **6**
- 5.1 Model 1: the rule encoder (this is what ships) . . . 6
- 5.2 Model 2: the learned encoder (research only, not shipped) . . . 7
- 5.3 Model 3: the cascade (best result, not yet shipped) . . . 7

6 Datasets **7**
- 6.1 Preprocessing . . . 8

7 Experimental results **8**
7.1 Message size . . . 8
7.2 Encoder accuracy by group (10,000 messages) . . . 8
7.3 Rules vs learned model vs cascade . . . 8
7.4 Network delivery . . . 9
7.5 Why not ordinary compression . . . 9
7.6 Testing . . . 9

8 What is incomplete, and how to fix it **9**
8.1 No real human-written test data . . . 9
8.2 Nothing has run on a real phone . . . 10
8.3 No confidence intervals on the network results . . . 10
8.4 The Hindi claim has no Hindi data . . . 10
8.5 The security tag is weak and has no threat model . . . 10
8.6 The cascade is measured but not deployed . . . 11
8.7 The fallback path is wasteful . . . 11
8.8 The radio model is not validated . . . 11
8.9 Old results in `docs/EVALUATION.md` are single-seed . . . 11

9 Roadmap **11**
9.1 Week 1 — make the numbers defensible . . . 11
9.2 Weeks 2–3 — real evaluation data . . . 12
9.3 Week 4 — improvements worth making . . . 12
9.4 Week 5 — paper . . . 12
9.5 Honest assessment of where it can go . . . 12

10 How to run everything **12**

1. What this project is

1.1 The problem

When an earthquake or flood hits, mobile towers stop working. People who need help cannot call anyone. The phones still work — they just have no network to reach.

Smartphone mesh networking solves this by letting phones talk to each other directly over Bluetooth. A message hops from phone to phone until it reaches someone who still has internet, who forwards it to a rescue centre.

1.2 The specific problem we work on

Bluetooth has a hard limit. In the mode that works on *every* phone — including cheap and old ones, which is what people actually carry in disaster-prone areas — one transmission carries only **31 bytes**. After the mandatory Bluetooth headers, about **27 bytes** are usable.

Our emergency message does not fit. Measured on our own code:

Message	Size	Transmissions needed
Heartbeat (“I am alive”)	25 B	1
SOS with no text at all	124 B	5
SOS with a short message	161 B	6

The surprise: the text is not the problem. An SOS with *zero* characters of text is still 124 bytes, because it carries a digital signature (64 B) and a public key (32 B). Cryptography is **96 of those 124 bytes — 77% of the message**.

This means compressing the text alone takes a message from 6 transmissions to 5. It can never reach 1.

1.3 Why splitting a message is bad

If a message needs 6 transmissions, all 6 must arrive or the message is lost. With 3% loss per transmission:

- 1 transmission: fails 3% of the time
- 6 transmissions: fails 16.7% of the time

It also keeps the radio busy about six times longer, and in a crowded mesh where every phone is relaying, that is what limits how many messages get through.

1.4 What we built

A path that fits a complete, authenticated emergency report into **one** 25-byte transmission:

1. The person types what happened, in English or Hindi or both.
2. The phone reads it and compresses the meaning into **3 bytes**.
3. Those 3 bytes go into a 25-byte message with location and a security tag.
4. If the phone did *not* understand the message, it sends the original text instead — slower, but nothing is guessed.

2. Where the project stands today

Part	Status	Note
3-byte semantic envelope	**Done**	Packs and unpacks, tested
Rule-based encoder (English + Hindi)	**Done**	0 KB, 0.1 ms per message
Decoder and triage scoring	**Done**	Replaces the old hardcoded weights
25-byte authenticated message	**Done**	Fits one frame, tag survives hops
Size-aware mesh simulator	**Done**	Airtime, fragmentation, collisions
Network evaluation (A/B/C)	**Done**	60 trials per cell
Stratified evaluation, 10,000 msgs	**Done**	Five strata
Learned model + cascade	**Done**	Trained, compared, reported
Compression baselines (gzip etc.)	**Done**	Answers the obvious objection
Android app with text input	**Done**	Builds; APK from GitHub Actions
Research paper (IEEE format)	**Done**	7 pages, 4 figures
Real human-written test data	**Not started**	Blocking a good venue
Measurement on a real phone	**Not started**	One day of work
Confidence intervals on results	**Not started**	A few hours
Hindi data collection	**Not started**	No corpus has it
Threat model for the security tag	**Not started**	One section
Cascade deployed in the app	**Not started**	Measured, not shipped

3. The application, explained

3.1 Who the actors are

- **Victim phone** — someone in trouble sends an SOS.
- **Relay phones** — ordinary phones nearby that pass the message on. They do this in the background, without anyone touching them.
- **Gateway phone** — a phone that still has internet. It uploads everything it has heard.
- **Rescuer dashboard** — a web map that shows where people need help.

3.2 What the user actually does

1. Opens the app. It is already running in the background, sending a small "I am alive" heartbeat every few seconds.
2. Types what happened — for example *"3 log malbe me fanse hain, ek behosh"*.
3. While typing, a line below the box shows what the mesh will actually carry. Green means understood and compressed; orange means not understood, full text will be sent.
4. Presses SOS. The message goes out over Bluetooth.

3.3 Technologies used

Part	Technology
Phone app	Kotlin, Android SDK 34, minimum Android 8
Radio	Bluetooth Low Energy, legacy advertising (works on all phones)
Background running	Android Foreground Service with `connectedDevice` type
On-device NLP	Pure Kotlin rules — **no model file, no TensorFlow, no internet**
Security	HMAC-SHA256 truncated to 4 bytes; Ed25519 for the full message
Server	Python, FastAPI, SQLite
Dashboard	HTML + JavaScript + Leaflet maps
Simulator	Python, discrete-event
Research models	scikit-learn (TF-IDF, logistic regression, MLP)
Build	GitHub Actions produces the APK automatically

3.4 How data flows, step by step

1. **Text in.** The user types into the app.
2. **Encode.** `SemanticEncoder.kt` reads the text, matches words against its lists, and produces six values plus a confidence score.
3. **Decide.** If confidence ≥ 0.45, compress. Otherwise send the original text.
4. **Pack.** `CompactSos.kt` builds a 25-byte message: header (13) + grid location (4) + battery (1) + envelope (3) + security tag (4).
5. **Sign.** An HMAC tag is computed over everything except the hop counter, so the tag still works after the message has travelled.
6. **Broadcast.** `BleTransport.kt` sends it as one Bluetooth advertisement.
7. **Relay.** Every phone that hears it checks whether it has seen it before, reduces the hop counter, waits a random moment so phones do not all shout at once, and re-sends.
8. **Store and forward.** If nobody is nearby, the phone keeps the message for up to 6 hours and re-sends every 10 seconds.
9. **Gateway.** A phone with internet unpacks the 3 bytes back into readable fields and uploads them.
10. **Dashboard.** The rescuer sees the location, what happened, how many people, and a triage score — plus a warning if the encoder was unsure.

3.5 What the 3 bytes contain

Field	Bits	Meaning
Category	4	trapped, collapse, medical, fire, flood, evacuation, supplies, rescue, other
Casualty count	3	0–6 people, 7 means “seven or more”
Mobility	2	unknown / can walk / cannot move / unresponsive
Hazard flags	6	fire, water, gas, structural, electrical, chemical
Severity	2	unknown / none / minor / serious
Confidence	3	how sure the encoder was, 0–7
Reserved	4	for future use

Confidence is sent *with the data* on purpose, so the rescuer can see which parts of a report were guessed.

4. Existing research and where the gaps are

4.1 What already exists

Work	What it does, and what it leaves out
HumAID, CrisisBench	Large labelled sets of disaster tweets. Solved problem. They label what a tweet is *about*, not what the emergency is.
ResQMesh (SSRN, 2026)	Closest work. On-device classifier in a BLE mesh app. It uses the category to *order* messages. Message size never changes.
OEPB (IETF draft, 2026)	Specifies authenticated emergency broadcast. Uses a 256-byte message and explicitly *accepts* splitting on Bluetooth 4.x.
RelayZero (blog, 2026)	Proposes the same idea as us, but with no evaluation, no radio (a TCP socket), and numbers that contradict each other.
Semantic communication	Active field, but assumes a channel with real capacity. Nobody targets 27 bytes.
CuMAC	Short security tags for small devices, built up over many messages. Prior art for our 4-byte tag.

4.2 The gaps we found

1. **Nobody targets the frame boundary.** Everyone either splits the message or assumes a bigger radio.
2. **Nobody measures the network effect.** Prior work reports classifier accuracy or queue latency, not delivery.
3. **Nobody confronts the crypto budget.** Because they do not target one frame, they never discover that 77% of the message is signature.
4. **The corpora collect the wrong people.** We sampled 520 tweets from HumAID: 51% commentary, 18% news, 17% retweets. Crisis datasets are written by *observers*; our system carries messages from *participants*.

4.3 What is new in our work

1. A complete **authenticated** report in one 27-byte frame, with every trade stated.
2. A mesh simulator that actually models message size, and delivery numbers from it.
3. The measured finding that cryptography, not text, is what breaks the budget.
4. Two honest negative results: a synthetic-trained model that looks excellent and is not, and a standard corpus that does not match the deployment.

5. The models

5.1 Model 1: the rule encoder (this is what ships)

No training, no model file, no internet. It works by matching words against lists.

- **Lexicons**: lists of words per field, in English and romanised Hindi together — `trapped`, `fansa`, `malbe ke neeche` all live in the same list.

- **Number extraction**: a digit counts only if a person-word appears within 28 characters after it, so "4 people trapped" is read and "4th floor" is not.
- **Negation**: English negates *before* ("no injuries"), Hindi negates *after* ("ghayal nahi"). Both sides are checked — 18 characters before, 14 after.
- **Confidence**: starts at (fields recovered)/5, then is reduced if most of the message was not recognised ($\times 0.6$), if the writer sounded unsure ($\times 0.8$), if no category matched ($\times 0.5$), or if the message is very short ($\times 0.7$).

Why this matters: if the encoder cannot read most of a message, its confidence goes *down*, not up. It treats its own confusion as a reason not to compress.

5.2 Model 2: the learned encoder (research only, not shipped)

Setting	Value and why
Features	TF-IDF over character n-grams, 2 to 4 characters
Vocabulary size	4,000 (script) / 3,000 (notebook)
Weighting	Sublinear term frequency
Classifier (script)	Logistic regression, `max_iter`=1000, `C`=4.0
Classifier (notebook)	MLP, one hidden layer of 128 units, 30 epochs
Heads	One per field; one binary head per hazard bit
Training size	6,000 generated messages
Training time	9.2 s (logistic) / 5.8 s (MLP), CPU only
Model size	1,435 KB float32, 808 KB int8
Speed	0.83 ms per message

Why character n-grams, not words: people spell romanised Hindi differently every time — *paani*, *pani*, *panee*. Character pieces survive that; whole words do not.

5.3 Model 3: the cascade (best result, not yet shipped)

Run the rules first. If they are confident, use their answer. If not, pass the message to the learned model. Nothing extra is trained — the routing signal is the confidence score that already existed.

6. Datasets

Set	Size	What it is and what it is for
Hand-written	25	Written by us. English and Hindi. A smoke test, not a benchmark.
Synthetic training	6,000	Built from templates. Labels are correct by construction, not copied from the rule encoder.
Stratified test	10,000	Five groups of 2,000: known words, synonyms the lexicon lacks, misspellings, code-mixed, and nonsense.
HumAID	76,484	Real tweets, downloaded. **Not usable as-is** — mostly commentary and news. English only.

The honest position: there is no real human-written test data in this project yet. That is the single biggest weakness.

6.1 Preprocessing

Deliberately minimal, because the encoder has to run on a phone in milliseconds:

1. Unicode normalisation (NFKC) — makes different ways of writing the same character identical.
2. Lowercase.
3. Collapse repeated whitespace.
4. **Nothing else.** No stemming, no stopword removal, no spell correction, no tokenisation into words. Matching is done on raw substrings so multi-word phrases like *"malbe ke neeche"* work.

7. Experimental results

Everything below was measured on this code and can be reproduced.

7.1 Message size

	Bytes	Frames
Full SOS with text	162–187	6–7
Compact SOS	**25**	**1**
Reduction		**about 86%**

7.2 Encoder accuracy by group (10,000 messages)

Group	Category	Count	All fields	Fell back
Known words	79.8%	100.0%	58.6%	16.2%
Synonyms (unseen)	8.3%	60.3%	0.1%	97.0%
Misspellings	69.8%	100.0%	50.9%	25.2%
Code-mixed	79.9%	100.0%	60.4%	17.4%
Nonsense	—	—	—	100.0%

Read the synonyms row. When the encoder meets words it does not know, it almost never gets the answer right — but it refuses to compress 97% of the time instead of guessing. It fails *slowly*, not *wrongly*.

7.3 Rules vs learned model vs cascade

Group	Rules	Model	Cascade
Known words	79.8%	62.3%	**81.2%**
Synonyms	8.3%	**39.1%**	38.8%
Misspellings	69.8%	61.7%	**74.8%**
Code-mixed	**79.9%**	50.5%	75.8%
Overall	59.5%	53.4%	**67.7%**

The model scored **99.7%** on data from its own generator and **28%** on hand-written text. It learned the generator, not the task. But it beats the rules by five times on unseen wording, so combining them beats either.

7.4 Network delivery

60 trials per cell, 3% loss per frame.

Nodes	Full text	Always compressed	Compressed if sure	Airtime saved
50	80.0%	88.3%	86.7%	15.7%
100	76.7%	85.0%	80.0%	12.0%
200	53.3%	60.0%	60.0%	8.5%

7.5 Why not ordinary compression

Method	Average	Worst	Fits 27 B?
Raw text	48.2 B	65	no
gzip	65.8 B	78	no
LZMA	105.3 B	124	no
zlib with shared dictionary	11.2 B	51	sometimes
Our envelope	**3.0 B**	**3**	**always**

Normal compressors make short messages *bigger*, because their own headers cost more than they save.

7.6 Testing

27 automated tests pass. One of them fails the build if anyone adds a field that pushes the message past 27 bytes.

8. What is incomplete, and how to fix it

This is the important section. Each item says what is wrong, why it matters, and exactly how to fix it.

8.1 No real human-written test data

Status: Not started. This blocks publication at a good venue.

Why it matters. Every accuracy number in this project comes from text that we or a generator wrote. A reviewer will say the evaluation measures our own writing style.

How to fix.

1. Run `notebooks/MeshSOS_build_annotation_set.ipynb` in Colab. It downloads HumAID and writes two labelling sheets.
2. **Filter first.** The notebook now counts how many messages are genuine first-person distress. If that number is small, use the *Disaster Response Messages* corpus instead — its `direct` genre is real SMS sent during the Haiti and Chile earthquakes.
3. Two people label 300–500 messages **independently**, following `docs/ANNOTATION_GUIDE.md`. Not together — agreement measured after discussion means nothing.
4. Run `python nlp/score_annotations.py A.csv B.csv`. It reports Cohen's kappa per field and lists disagreements.
5. Settle the disagreements, build `gold.csv`, then run the same script with `--evaluate`.

Effort: 6–8 hours per person, two people.

8.2 Nothing has run on a real phone

Status: Not started. Second biggest weakness.

Why it matters. All latency figures are desktop Python. There is no battery measurement. For a paper about phones, that is a visible hole.

How to fix.

1. Install the APK (already built by GitHub Actions) on one cheap Android phone, four years old or more.
2. Add timing around the `encode()` call — use `System.nanoTime()` and log it.
3. Type 50 messages, record the times, report mean and worst case.
4. Note the APK size difference with and without the encoder.
5. If time allows, use Android's Battery Historian for drain over an hour.

Effort: one day. **This is the cheapest big improvement.**

8.3 No confidence intervals on the network results

Status: Not started.

Why it matters. The 100-node result is +4.3%. With 60 trials and no error bars, nobody can tell whether that is real or noise. A reviewer will write exactly that.

How to fix. In `simulator/eval_compression.py`, record each trial's outcome instead of only the total, then report mean ± 95% confidence interval, and run a two-proportion test between A and C. Add error bars to the delivery figure.

Effort: 2–3 hours.

8.4 The Hindi claim has no Hindi data

Status: Not started.

Why it matters. The paper claims code-mixed support and reports 60.4% on it — but that is on messages we generated. HumAID is English only. No public disaster corpus has Hindi.

How to fix. Give 25–30 classmates a scenario — *"imagine you are trapped after an earthquake; write the message you would send"* — and collect 5 each. That is 150 messages in a day, written by people who are not the authors. Describe it in the paper as *elicited*, not collected in the wild. Honest and acceptable.

Effort: one day.

8.5 The security tag is weak and has no threat model

Status: Partly — the tag works, the analysis does not exist.

Why it matters. A 4-byte tag means a 1-in-4-billion forgery chance per attempt. The paper says so but never says who the attacker is or what they can do.

How to fix.

1. Write a short threat-model subsection: who the attacker is, what they can send, what they cannot do, and what the gateway does about rate limits.
2. **Better:** implement the CuMAC idea. Split a full 16-byte tag into four 4-byte pieces and send a different piece in each re-broadcast. The store-and-forward layer already repeats every 10 seconds for 6 hours, so the repeats are free. The gateway acts on the first frame and reaches full strength over later ones.

Effort: 2 hours for the section, 2 days for the implementation.

8.6 The cascade is measured but not deployed

Status: Partly.

Why it matters. The best result in the paper (67.7%) is not what the app does. The app runs rules only.

How to fix. Either ship it — which means putting a quantised model in the APK and adding TensorFlow Lite — or state clearly in the paper that it is an offline result. We currently do the second, which is honest. Shipping it is a real engineering job because it adds a model file and a dependency.

Effort: 3–4 days to ship properly.

8.7 The fallback path is wasteful

Status: Not started — a known easy improvement.

Why it matters. When the encoder is unsure (23% of messages), it sends raw text — 162–187 bytes, 6–7 frames. Our own measurements show shared-dictionary compression would bring that to about 11 bytes.

How to fix. Build a dictionary from common emergency phrases, ship it with the app, and compress the fallback text with `zlib` using that dictionary. Both ends must have the same dictionary.

Effort: one day.

8.8 The radio model is not validated

Status: Partly.

Why it matters. We wrote the collision and airtime model ourselves. It has not been checked against a real measurement or a second simulator.

How to fix. Either compare against a known BLE simulator, or measure real airtime on two phones with a sniffer, or state it clearly as a limitation (currently done).

Effort: days to weeks. Lowest priority of the list.

8.9 Old results in `docs/EVALUATION.md` are single-seed

Status: Partly — known and documented.

Why it matters. The original 200-node "0 delivered" figure came from one unlucky random seed. The median across seeds is 6 hops, not 17.

How to fix. Re-run `simulator/evaluation_matrix.py` over 25 seeds and regenerate the file with mean and standard deviation.

Effort: one hour.

9. Roadmap

9.1 Week 1 — make the numbers defensible

Day	Task	Output
1	Confidence intervals and significance tests on all network results	Updated tables, error bars
1	Re-run old evaluation over 25 seeds	New `EVALUATION.md`
2	Install APK on one old phone; time the encoder; note APK size	Real latency numbers
2	Write the threat-model subsection	New § in the paper
3–5	Collect 150 Hindi messages from classmates	First real data

9.2 Weeks 2–3 — real evaluation data

1. Decide the corpus: HumAID filtered, or Disaster Response Messages `direct` genre. Read samples from both before choosing.
2. Build the annotation sheets in Colab.
3. Two people label 300–500 messages independently.
4. Score agreement; settle disagreements; build the gold set.
5. Re-run every experiment against it. The scripts already exist — only the data changes.

9.3 Week 4 — improvements worth making

1. Compress the fallback path with a shared dictionary.
2. Implement segmented authentication over the store-and-forward repeats.
3. Optionally ship the cascade in the app.

9.4 Week 5 — paper

1. Replace every synthetic number with the real one.
2. Add the hardware measurements.
3. Add error bars to all figures.
4. Re-read the Related Work; check whether the FMEC 2026 ResQMesh paper has appeared.
5. Have someone outside the team read it for clarity.

9.5 Honest assessment of where it can go

Venue	Today	After the roadmap
College / university symposium	Strong	Strong
National conference	Good	Strong
IEEE / ACM workshop	Possible	Good
Mid-tier conference	Unlikely	Possible
Top venue (SenSys, MobiCom)	No	Still unlikely

The two items that move this the most are **real data** and **one phone measurement**. Everything else is polish.

10. How to run everything

```
pip install -r requirements.txt

python demo.py                                  # end-to-end walkthrough
python -m pytest tests/ -q                      # 27 tests
python simulator/test_phase0_size_aware.py      # size-awareness check
python nlp/eval_encoders.py 2000                # 10,000-message evaluation
python nlp/train_model.py                       # the learned model
python simulator/eval_compression.py 60         # network results
python simulator/make_figure.py                 # delivery chart
python paper/make_figures.py                    # all paper figures
python simulator/ttl_vs_size_experiment.py 25   # TTL vs size
```

Backend and dashboard: `./start.sh` or `docker compose up`.

The Android APK is built automatically by GitHub Actions on every push; download it from the workflow run's artifacts.

One last thing. Every number in this report came from running the code in this repository. If anyone asks where a figure came from, the command that produced it is listed above. That is worth more than any single result.